For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Caching Case Study - 3

Facebook News Feed

Situation

`~3 Billion` Monthly Active Users (MAU)
Users can make posts on someone's profile
Facebook earns via ads — shows people relevant content in their newsfeed

Each `post` has

- an `author`: who made the post
- a `recipient`: whose profile was this post made on
- content

Note: the user can post on their own profile (in this case, the author and the recipient will be the same)

Facebook maintains a friendship graph

- users can be friends with other user
- on average each person has ~1000 friends in the facebook social graph

Facebook DB Schema (simplified)

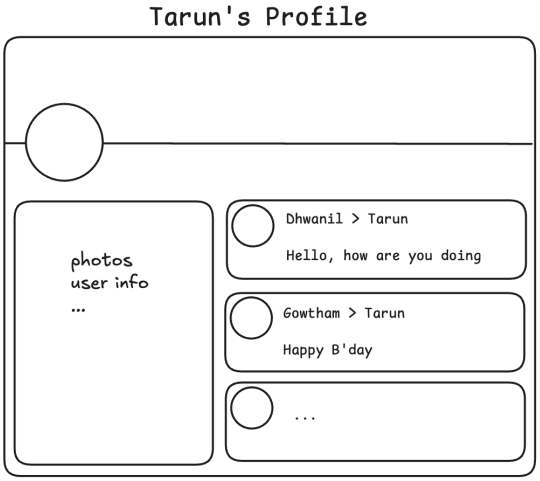
```
users
id      name     gender    avatar_url   ...

user_friends
user_1_id      user_2_id     created_at    ...

user_posts
author_id    profile_id     content
```

1. **User Profile**

Shows any posts that were made by any author on the user's profile

example: Tarun's profile will show all posts made by (Tarun or his friends) on Tarun's profile

```
select * from user_posts
where profile_id = [user_id]
```



2. **User Newsfeed**

   Any relevant, recent posts made by any of the user's friends (or people that the user follows)

   example: Tarun's newsfeed will show all recent posts made by any of Tarun's friends

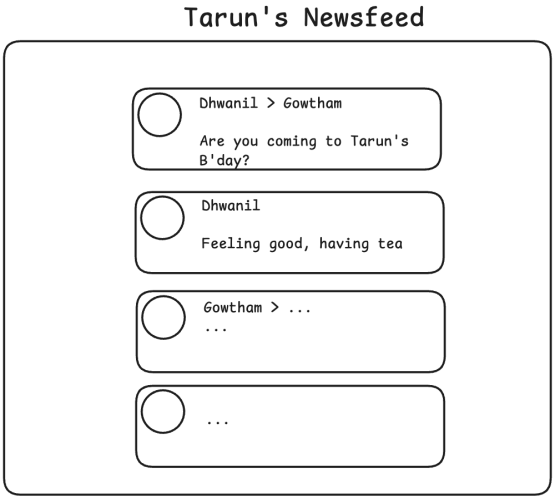
   a. find all recent posts from all the friends of the user

   ```
   select * from user_posts
         where profile_id in (
               select user_2_id from user_friends
               where user_1_id = [user_id]
         )
   where updated_at > now() - 30 days
   ```

   b. calculate how relevant this post is to this user
      - News factor
      - Recency
      - Shock value
      - user-user affinity
      - geo-location

   c. rank the posts by the relevancy score

   d. paginate the data to show the newsfeed



Scale Estimation

Back of the Envelope/Napkin calculations
Approximate calculations => guesstimation

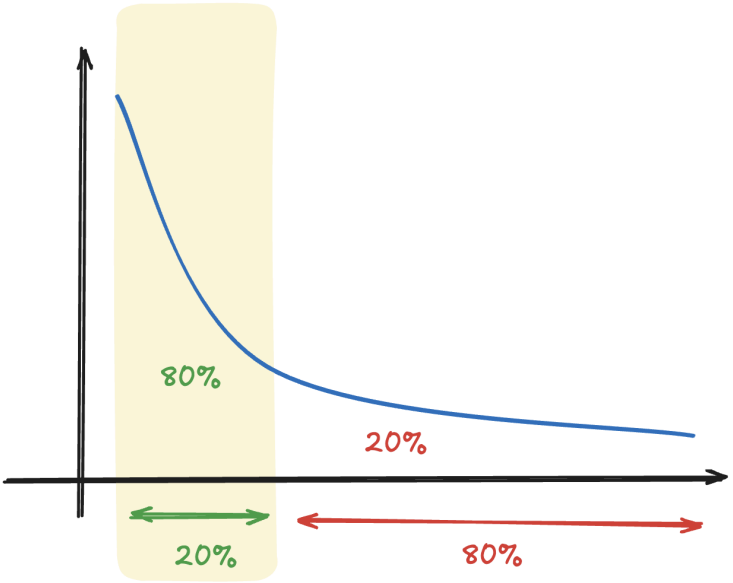
Goal is NOT *"to be accurate"*
Goal is *"to quickly get a sense of the scale"*

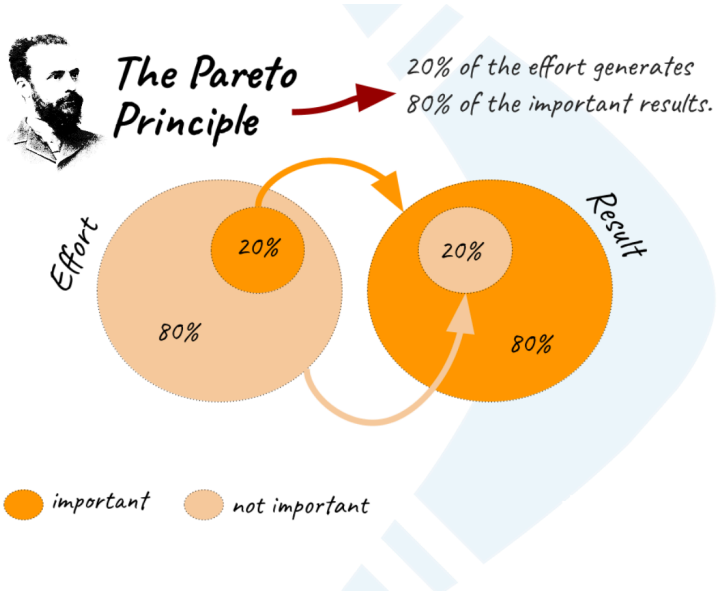
Monthly Active Users (MAU)**:** `~3 Billion`
Daily Active Users (DAU)**:** `~2 Billion`
Number of people creating posts:

for **social media** we can extend the Pareto principle (80-20)
to the 80-20-1 principle

- 80% of the users are going to be passive watchers
  - they will just scroll through the news feed, watch reels, …
- 20% of the users will be active watchers
  - they will interact with the content (comment, like, follow)
  - they will not create the content frequently
- 1% of the users will be active content creators
  - the ones the actually make the posts!

`= 1% of 2 Billion`
`= ~20 Million`





Avg posts / active user / day = 5
Number of posts / day:
(5 posts / active user / day) * (20 M active users)
= 100 Million posts / day

Total number of posts over 20 years:
(100 Million posts / day) * (20 years)
= (100 Million posts / day) * (20 * 365 days)
= ~800 Billion posts

Size of each post:
- post_id: *uuid (16 bytes)*
- author_id: *uuid (16 bytes)*

- profile_id: *uuid (16 bytes)*
- content: *text (avg. 100 bytes)*
- attachment_url: *text (avg. 200 bytes)*
- created_at: *timestamp+tz (16 bytes)*
- updated_at: *timestamp+tz (16 bytes)*
- location: *geolocation (16 bytes)*
- feeling: *enum (1 byte)*

Each post is `500 bytes` on average

Total amount of post data (over 20 years)**:**

$= (800 \text{ billion posts}) * (500 \text{ bytes / post})$

$= (8 * 10^{11} \cancel{\text{posts}}) * (5 * 10^{2} \text{ bytes / } \cancel{\text{post}})$

$= 40 * 10^{13} \text{ bytes}$

$= 400 \text{ TB}$

Need for caching

Q: Does all the data fit on a single machine?

Nope.
With modern tech, it might be possible to store all this in a single server
But a single server won't be able to handle all the queries for billions of users.

Even if I can fit it on a single server, that server will become a bottleneck and a single point of failure.

Instead, I should shard & replicate the data and store it across multiple servers.

Q: What is the ideal sharding key?

The ideal thing is to shard by **`user_id`**
This means that each user's data lives completely inside a single shard

1. all posts made by that user on theirs or someone else's profile
2. all posts made on that user's profile

For example, we've the following posts

- P1: Mukesh → Shubhadeep: Hi, how are you
- P2: Shubhadeep → Mukesh: Doing fine, wassup with you?
- P3: Dinesh → Mukesh: we're both …kesh
- P4: Vinetha → Harshit: Happy B'day
- P5: Harshit → Mukesh: bro, today's my B'Day

In this case, we will have the following data in the various shards

- Mukesh: `P1, P2, P3, P5`
- Shubhadeep: `P1, P2`

Note that I'm not saying that every user has a different shard/server. A single server will store the data for multiple users..

- Server 1: stores the data for Mukesh, and Dinesh
  - user_posts
    - P1: autor_id = Mukesh, profile_id = Shubhadeep, content = Hi how are you
    - P2: …
    - P3: …
    - P5: …
- Server 2: Vinetha and Shubhadeep's data
  - user_posts
    - P1
    - P2
    - P4

Notice how the same post is automatically replicated across 2 shards (author's shard, and the recipient's shard). Note that the author and recipient for some post might live inside the same server.

Q: How do we decide which user goes to which shard?

Consistent hashing based on user_id
Why user_id? Why not initial letter of name?
Because initial letter of name or geolocation will be a bad sharding key/
How to choose a good sharding key? We will discuss this in lecture 7

Q: If we're sharding by user_id, and there's 3 billion users, does that mean that we will have 3 billion shards or 3 billion servers?

No, each server will store the data for millions of users.

Let's image that each server comes equipped with 1TB of hard disk.
If each post is 500 bytes in size, the each server can store approximately

```
1TB / (500 bytes / post)
= 1 * 10^12 bytes / (500 bytes / post)
= 2 billion posts
```

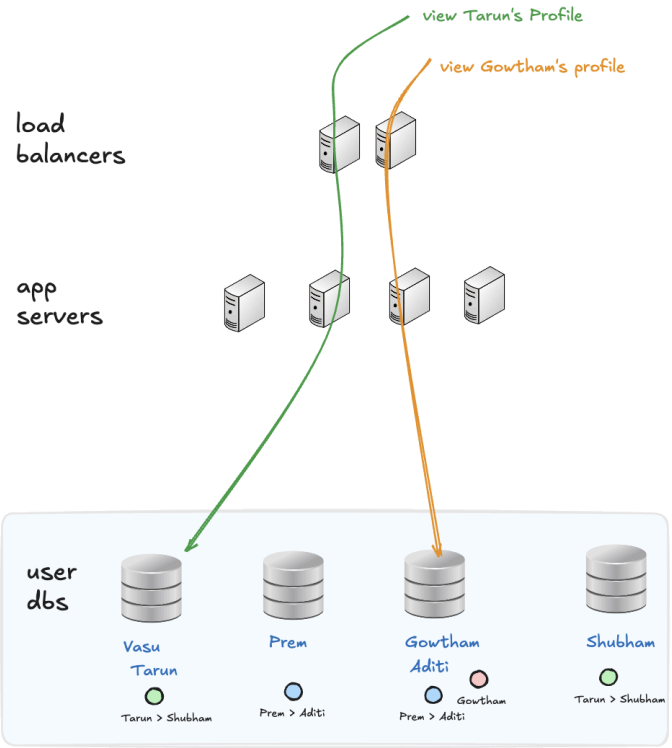
Average number of posts made by a user over their lifetime
Let's say `5000` just to upper bound it

The number of users a single server can accommodate will be

```
(2 billion posts / server) / (5000 posts / user)
= 2 billion / 5000   (users/server)
= 400,000 users / server
```

Given this, fetching the contents to view someone's profile is easy

example: if I want to view Tarun's profile, I can just go to Tarun's shard, since all posts made on Tarun's profile (by his friends, or himself) live inside Tarun's shard

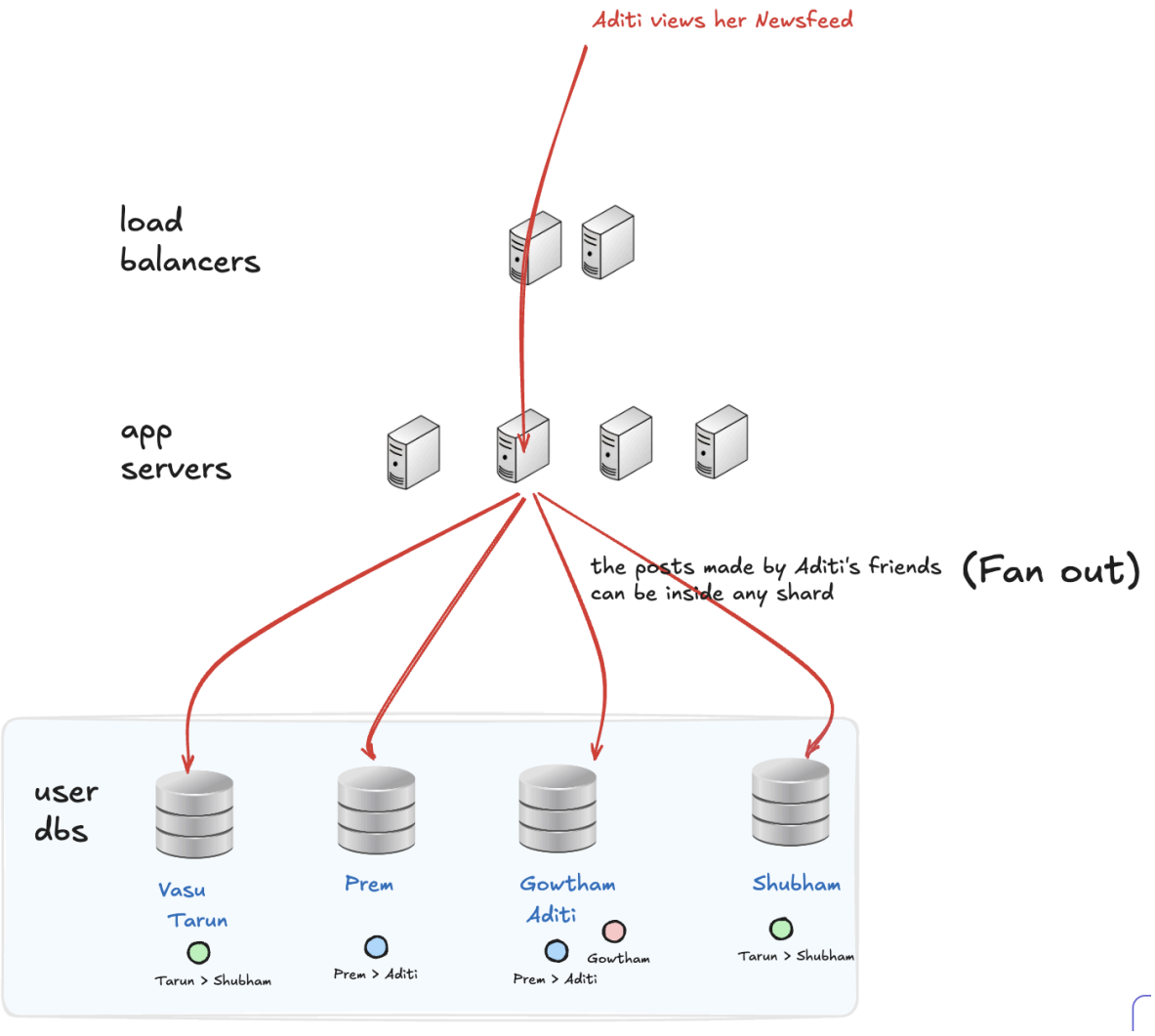


Q: What data do we need to compute the newsfeed?

We need all ***recent*** posts made by all friends of the user
(posts made by user's friends on anyone's profile)

Q: How to retrieve the data for calculating a user's newsfeed?
That will be a fan-out read
Because the user's friends can be inside any shard, to get all their posts, we will need to hit every possible shard - ***this is extremely expensive***

Hence, we need caching!



Caching the Newsfeed for all users

Number of posts / newsfeed page: `10 posts / page`
Max newsfeed pages: `50 pages`
after user has scrolled down through 50 pages, we will show something like *"you're all caught up"*

Total Newsfeed size:
`= (500 bytes / post) * (10 posts / page) * (50 pages / user) * (2 billion users)`
`= 50 * 10,000 * billion bytes`
`= 500 TB`

Is it possible to cache the Newsfeed for all users?

No! There's just too much data to cache if we want to precompute and cache the entire newsfeed for all the users!

In fact, it's not just that

1. every time any makes a post, it will invalidate the newsfeed of all of their friends!
   on avg each user has ~1000 friends
   content creators have ~5000 friends, and the content creators are the ones making 80% of the posts
2. the newsfeed ranking algorithm changes frequently
   the dev team might do 4-5 updates to this ranking algorithm each day
   every time they do an algorithm change, this invalidates the entire 500TB of newsfeed cache!

We cannot pre-compute & cache the newsfeed for all the users!

How can we optimize Newsfeed calculation?

Q: To calculate the news feed for a user, do we care about posts that were made 5 years ago by their friends?

No! We only care about the recent posts.

Q: How "old" should a post be to not show up in the newsfeeds?

let's say 30 days

Q: How large is 30 days worth of recent post data?

```
(100 million posts / day) * (500 bytes / post) * (30 days)
= 15 * 10^5 million bytes
= 1.5 TB
```

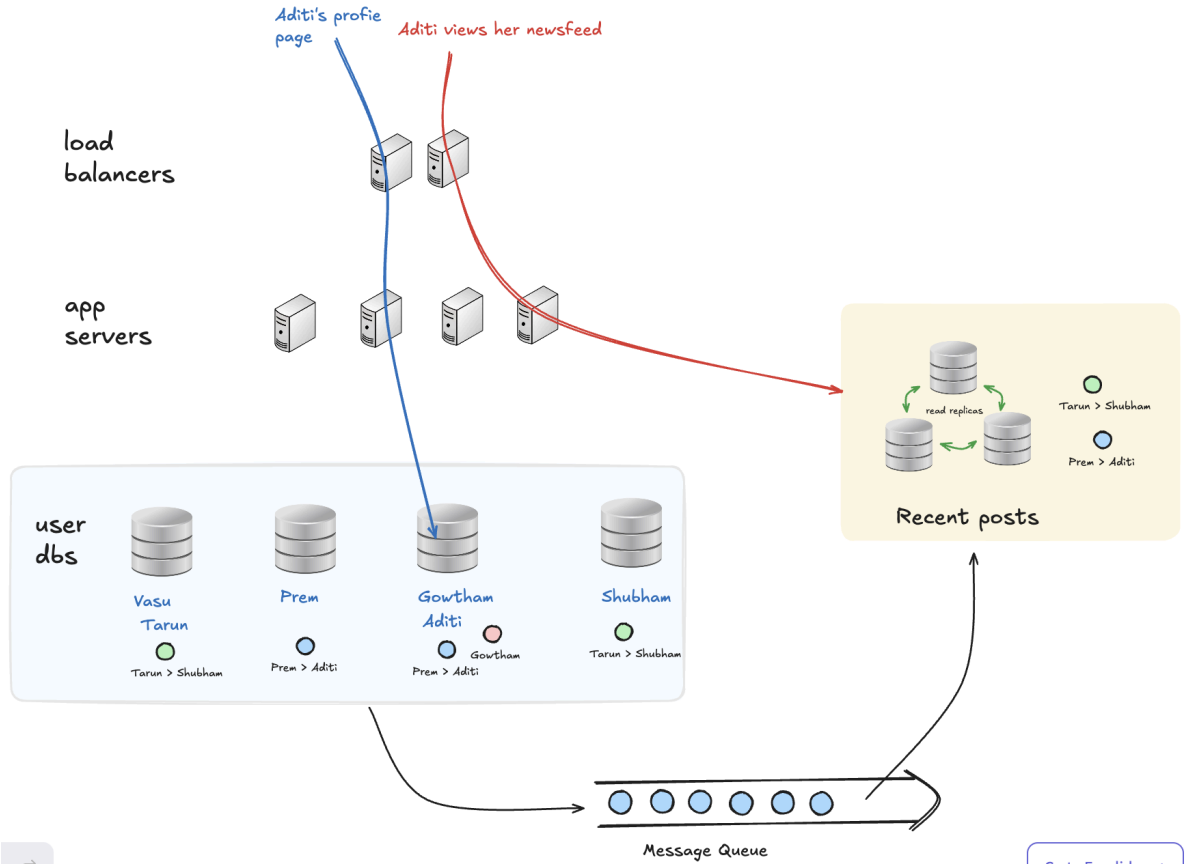
Can 1.5 TB of data fit on a single server? Easily (on the HDD)
So there's no need to shard the recent posts DB

Q: Will a single server be able to handle all the read requests for the newsfeed?

No worries, we can just have ***multiple read-replicas*** of that DB

If this is the case, then we can just cache the "recent" posts in a separate db

1. whenever a post is made, it is stored in the appropriate users' shards (both author and recipient)
2. the post is also cached in a separate "recent posts" db
3. any queries to view the profile page still go to the user's shard
4. any queries to view the newsfeed now no longer require a fan-out as they can just load the data from the recent-posts DB



Schema for Recent Posts db

every db (the user db shards, the recent posts db) have the same schema

```
users
id      name     gender ...

user_friends
user_1_id     user_2_id     created_at

user_posts
author_id    profile_id     content
```

Eviction Policy

Not needed, because the recent post data is not huge (1.5TB)

Invalidation Policy

A simple cron job that runs at midnight that removes posts that were made more than 30 days ago

```
delete from user_posts
where updated_at > (now() - "30 days")
```

Mandatory Reading (if applying for SDE 2+)

Scaling with Redis - Eviction policies and cluster mode

(Not reading material)

Framework Benchmarks (software only)

- Backend: https://www.techempower.com/benchmarks/
- Frontend: https://krausest.github.io/js-framework-benchmark/

There's only 2 hard things in computer science

1. naming things
2. cache invalidation

There's only 2 hard things in computer science

1. naming things
2. cache invalidation
3. off by one errors

There's only 3 hard things in computer science

4. thread synchronization
1. naming things
2. cache invalidation
3. off by one errors